

Serverless/Event-driven Architectural Design Report

Le Bao Ngoc
Swinburne University of Technology
Ho Chi Minh City
106216094@student.swin.edu.au

Tran Quoc Toan
Swinburne University of Technology
Ho Chi Minh City
106215606@student.swin.edu.au

Dao Le Hong Duc
Swinburne University of Technology
Ho Chi Minh City
105993820@student.swin.edu.au

Hoang Le Quang Anh
Swinburne University of Technology
Ho Chi Minh City
106218605@student.swin.edu.au

Abstract - This report extends the photo application developed in Assignment 2 by proposing a serverless, event-driven cloud architecture to support its increasing demands. As user adoption and system usage increases, the existing architecture faces challenges associated with scalability, high database operational costs and cross-region latency. To improve upon the present application infrastructure, the proposed architectural design adopts managed Amazon Web Services (AWS) including AWS Lambda, exposing APIs through Amazon API Gateway and securing authentication with Amazon Cognito. Media assets are retained in Amazon Simple Storage Service (S3) and delivered globally via Amazon CloudFront, with edge protection provided by AWS Web Application Firewall (WAF). Media uploads then initiate event-driven processing through Amazon SNS fan-out into per-type SQS queues, allowing independent workers to perform thumbnail generation, video transcoding, and future AI-based image analysis with Amazon Rekognition. The proposed architecture represents a migration from a traditional server-based model to a cloud-native architecture that satisfies growing traffic demands and adapts to unpredictable workload changes. A three-year growth forecast from 20,000 to 320,000 users indicates the deployment is stable and commercially feasible under a 3-video, 5-minute cap.

Keywords - *event-driven architecture, serverless model, target-tracking scaling, cloud migration, asynchronous fan-out, edge caching, extensible microservices*

I. INTRODUCTION

The Photo Album application developed in Assignment 2 established a highly available cloud-based system for users to upload, store, and access photos. The architecture implemented Amazon EC2 instances across multiple Availability Zones, an Elastic Load Balancer, an Auto Scaling Group, Amazon S3 for media storage, Amazon RDS for photo metadata, and AWS Lambda for thumbnail generation. This design improved availability and reliability while allowing the system to respond automatically to changes in workload demand. However, as the application continues to grow, further architectural improvements are required to support increasing usage, reduce database operational overhead, and enable future media capabilities.

The existing architecture relies on EC2-based compute and a relational database. Although the Auto Scaling Group provides a level of elasticity, running and maintaining application servers still requires infrastructure management and resource planning. As users and uploaded media increase, this approach results in higher operational cost and additional scaling complexity, while the relational database is not the most suitable solution for handling growing volumes of simple media metadata.

The objective of this report is to design and justify an AWS-based architecture that addresses the core pillars of the AWS Well-Architected Framework. The proposed solution provides a foundation that supports continued growth while reducing infrastructure requirements through managed cloud resources. The remainder of the report presents the architecture and its service-level rationale (Section II), representative use case flows (Section III), the design justification and cost evaluation (Section IV), and the assumptions, limitations and future directions of the design (Section V).

II. ARCHITECTURE DESIGN

A. High-Level Architecture Overview

Fig. 1 presents the complete cloud design proposed for the PhotoAlbum platform. The architecture reacts to events rather than polling for work, and every component is built to fail or fall behind without dragging the rest of the system down with it. Three principles guided the design: administration should be reduced to the minimum a managed service allows; components should stay decoupled through queues so that a surge or a failure in one pipeline never affects another; and capacity should follow demand automatically, since traffic is expected to double every six months for the next two to three years.

These principles produce four logical layers. The global edge layer, formed by Route 53, AWS WAF and CloudFront, shortens the distance to the user while filtering hostile traffic. The presentation and API layer delivers the static frontend from S3 and routes dynamic requests through API Gateway into Lambda, with Cognito supplying identity. The event and processing layer

- **Amazon API Gateway** is the managed entry point for the REST interface. It terminates TLS, enforces the Cognito authorizer, applies throttling, and invokes the correct Lambda function, drawing capacity from a shared pool that absorbs spikes without waiting for instances to launch.
- **AWS Lambda (App API)** carries the business logic: listing albums, creating and deleting records, searching, reporting processing status, and issuing pre-signed S3 URLs that let the browser upload directly to the raw bucket. It charges only for execution time and scales per request, which suits bursty, unpredictable traffic.

3) Storage and Data:

- **Amazon S3 (raw media bucket)** receives original uploads in whatever format the user supplies. It is the ingestion point and the trigger source for all downstream processing, since S3 emits an ObjectCreated event for every new object with no polling code involved.
- **Amazon S3 (derivatives bucket)** holds everything the workers produce: thumbnails, mobile-resolution images and transcoded video. Separating derivatives from originals allows different lifecycle rules and lets worker roles hold read access to one bucket and write access to the other, never both on the same bucket.
- **Amazon DynamoDB** stores metadata only, never the media itself: owner and album, the S3 keys of the original and its derivatives, a processing status field, and tags from image analysis. The access pattern is a lookup by identifier or a query by owner, with no joins and no ad hoc reporting, which is exactly where a key-value store outperforms a relational engine. On-demand capacity removes the fixed cost of an always-running instance and scales throughput automatically.

4) Event Distribution and Decoupling:

- **Amazon SNS** receives the S3 event and publishes it to every subscribed queue, filtering by content type to separate images from video. This fan-out is what makes the pipeline extensible: a future capability such as watermarking is added by subscribing a new queue to the existing topic, without touching the bucket configuration or any existing pipeline.
- **Amazon SQS** provides one queue per media type. The queue absorbs bursts, so simultaneous uploads lengthen a backlog rather than causing a failure, and each pipeline progresses at its own pace. A dead-letter queue collects repeatedly failing messages, so one corrupt file never blocks the pipeline behind it.
- **Amazon EventBridge** carries completion events outward to interested consumers once processing finishes, notifying the frontend to refresh while keeping notification separate from the media pipeline itself.

5) Processing Workers:

- **Image processing functions** consume from the image queue and produce thumbnails, mobile-resized versions and reformatted output, writing results to the derivatives bucket and updating status in DynamoDB. Lambda suits this work since it completes in seconds and fits comfortably within function memory limits.
- **EC2 Auto Scaling Group (video transcoding)** runs FFmpeg-based workers inside private subnets across multiple Availability Zones, long-polling the video queue and writing output to the derivatives bucket. Video runs on EC2 rather than Lambda because Lambda enforces a hard limit of fifteen minutes and roughly ten gigabytes of memory, which a long or high-resolution file can exceed. This is a deliberate match of platform to workload rather than a gap in the serverless design; MediaConvert and Fargate were considered as serverless alternatives and remain viable substitutes without changing the queue contract.

Choosing EC2 for this one workload is a deliberate trade-off rather than a default. It buys unbounded execution time and full control over the FFmpeg pipeline, but it reintroduces exactly the operational burden the rest of the design avoids: an AMI to patch, an idle baseline of at least two instances to absorb the first job before scaling reacts, and a component that, unlike Lambda, is billed by the hour rather than by the millisecond of actual use. The design accepts this because it is confined to a single, well-bounded pipeline — the queue-driven scaling policy keeps the idle cost small, and the video branch can be swapped for MediaConvert or Fargate later without touching any other component, so the trade-off is contained rather than structural.

- **VPC endpoints** let the transcoding fleet reach S3 and SQS over the AWS private network instead of a NAT gateway, keeping media traffic off the public internet and avoiding NAT data-processing charges.
- **Amazon Rekognition** is included as a planned extension: a dedicated queue and Lambda function would call the label detection API and write the resulting tags back into DynamoDB. The branch produces no new files, only metadata, so it terminates at the database rather than storage, demonstrating that the fan-out design accommodates new capability without disturbing what already exists.

6) Monitoring, Governance and Security:

- **Amazon CloudWatch** collects metrics and logs and drives the scaling policy for the video fleet. The metric chosen is the number of visible messages in the video queue rather than processor utilisation, because queue depth measures waiting work while utilisation only measures work already underway, letting the fleet scale before it saturates.

- **AWS CloudTrail and AWS CloudFormation** record every API call made within the account for audit purposes and define the stack as code, so environments are reproducible and every change is reviewed in version control rather than applied by hand. The deployment pipeline publishes the frontend build to S3 and updates function code automatically.
- **AWS IAM** governs access with a clear split between roles attached to services and groups assigned to people. Every function and instance profile receives its own role scoped to named resources, so an image worker can read the raw bucket and write the derivatives bucket but cannot delete either, and cannot see the video queue. Human access uses an administrator group with enforced multi-factor authentication, a developer group able to deploy and read logs but not alter policies or delete buckets, and a system manager group able to adjust scaling and drain failed jobs without reading customer media.
- **AWS KMS and AWS Organizations** encrypt both media buckets and the metadata table at rest with centrally managed, rotating keys, and separate development, staging and production into distinct accounts under service control policies that an individual account administrator cannot override.

C. Fulfilling the Business Scenario

The value of the design is easiest to judge against the pressures the business actually faces rather than against a list of features, so this section groups the reasoning into the concerns that drove the brief: growth, cost, reach, extensibility, and governance.

1) Absorbing growth without re-architecting:

Traffic doubling every six months over three years implies roughly a sixty-fourfold increase in load, and the current system already exceeds eighty percent processor utilisation on six small instances. The design responds in two ways. First, it removes the web and API tier from instances entirely: API Gateway and Lambda draw capacity from a managed pool and absorb a spike in seconds rather than waiting minutes for new servers to launch. Second, it places a queue in front of every remaining bottleneck, so a surge lengthens a backlog instead of dropping work. The video transcoding fleet, the one component still running on instances, is governed by a target-tracking policy driven by queue depth rather than processor use, adding capacity before the fleet saturates and holding steady-state utilisation in the intended range. The remaining components scale without any planning: object storage and the metadata table have no provisioned capacity to outgrow, functions scale per invocation, and queues have no practical depth limit.

2) Paying for use rather than for capacity:

The existing relational database is slow, costly, and structurally simple, which together argue against keeping it. A relational engine bills for an instance that runs whether or not anyone uses the application, and its strengths — joins and ad hoc queries — are strengths this workload never draws on. DynamoDB in on-demand mode charges per request, replicates across Availability Zones without configuration, and answers a key-based lookup in single-digit milliseconds at any scale. The same principle runs through the rest of the design: Lambda charges only for execution time, static assets are served from object storage rather than a running web server, and raw originals move to colder storage classes on a lifecycle rule. The transcoding fleet is the one place the business pays for idle capacity, and it is kept to a small baseline precisely because the queue allows a short delay under load rather than requiring capacity provisioned for the peak.

3) Serving a global audience from a single region:

Users outside Australia currently wait for every request to cross the ocean to Sydney. Rather than duplicate the application into several regions, which multiplies both cost and operational complexity, the design keeps a single write region and pushes reads to the edge. CloudFront caches the application bundle and the processed media at edge locations worldwide, so the first request in a region populates the cache and every subsequent request is answered locally. Route 53 resolves each user to the nearest healthy edge, and Origin Access Control ensures the buckets behind CloudFront are never publicly reachable, so the caching layer cannot be bypassed. A multi-region active deployment remains available later if the audience distribution changes, and the queue-based processing design would not need to change to accommodate it.

4) A processing pipeline built to change:

The brief anticipates video today and other media capabilities later, so the processing layer assumes new work will keep arriving. Uploads trigger processing automatically, since S3 emits an event on creation with no scheduler or watcher process involved. That event is published to a topic rather than delivered to a single consumer, which means an additional capability is introduced by subscribing a new queue and writing a new worker while every existing component stays untouched. Separating the queues also lets each kind of work run where it belongs: image processing is short and light, so it runs on Lambda, while video transcoding regularly exceeds Lambda's limits and runs on instances sized for sustained processor load. Because the two pipelines share nothing but the topic, a backlog of video has no effect on thumbnail generation, and a failure in one cannot propagate to the other.

5) Governing access and change:

Reducing administration does not mean reducing control. Service roles are attached to functions and instance profiles, each granting only the actions that component performs against named resources, while human access is organised into groups whose boundaries follow responsibility rather than convenience. Around this, KMS provides encryption at rest, CloudTrail records every

API call for audit, and CloudFormation defines the whole stack as code so that changes are reviewed in version control. Together these make the system's behaviour reproducible and its history auditable as the platform grows.

III. USE CASE COLLABORATION DIAGRAM

A. Media Upload Flow

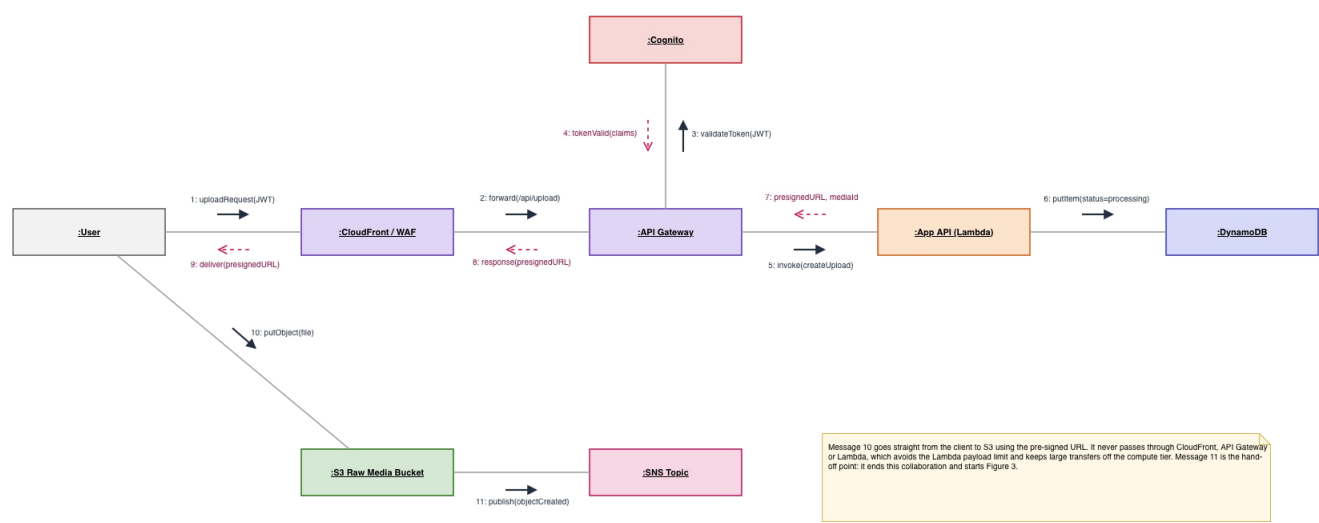


Fig. 2. Media Upload Flow.

The upload flow begins when an authenticated user initiates an upload from the client application. The request travels through Route 53 and AWS WAF to CloudFront, and from there to API Gateway, which has already validated the user's JSON Web Token against the Cognito authorizer before the request is allowed any further. API Gateway invokes the App API Lambda function, which writes an initial metadata record into DynamoDB with a processing status and generates a pre-signed S3 URL scoped to the raw media bucket. That URL, together with the metadata identifier, is returned to the client through the same path.

The client then performs the actual file transfer directly against the pre-signed URL, placing the object straight into the S3 raw media bucket. This step deliberately bypasses CloudFront, API Gateway and Lambda entirely, since routing a large image or video through the compute tier would be both slow and unnecessarily costly. The moment the object is written, S3 emits an ObjectCreated event, which is picked up by the SNS topic and marks the end of the upload flow and the beginning of processing.

B. Media Processing Flow

B. Media Processing Flow

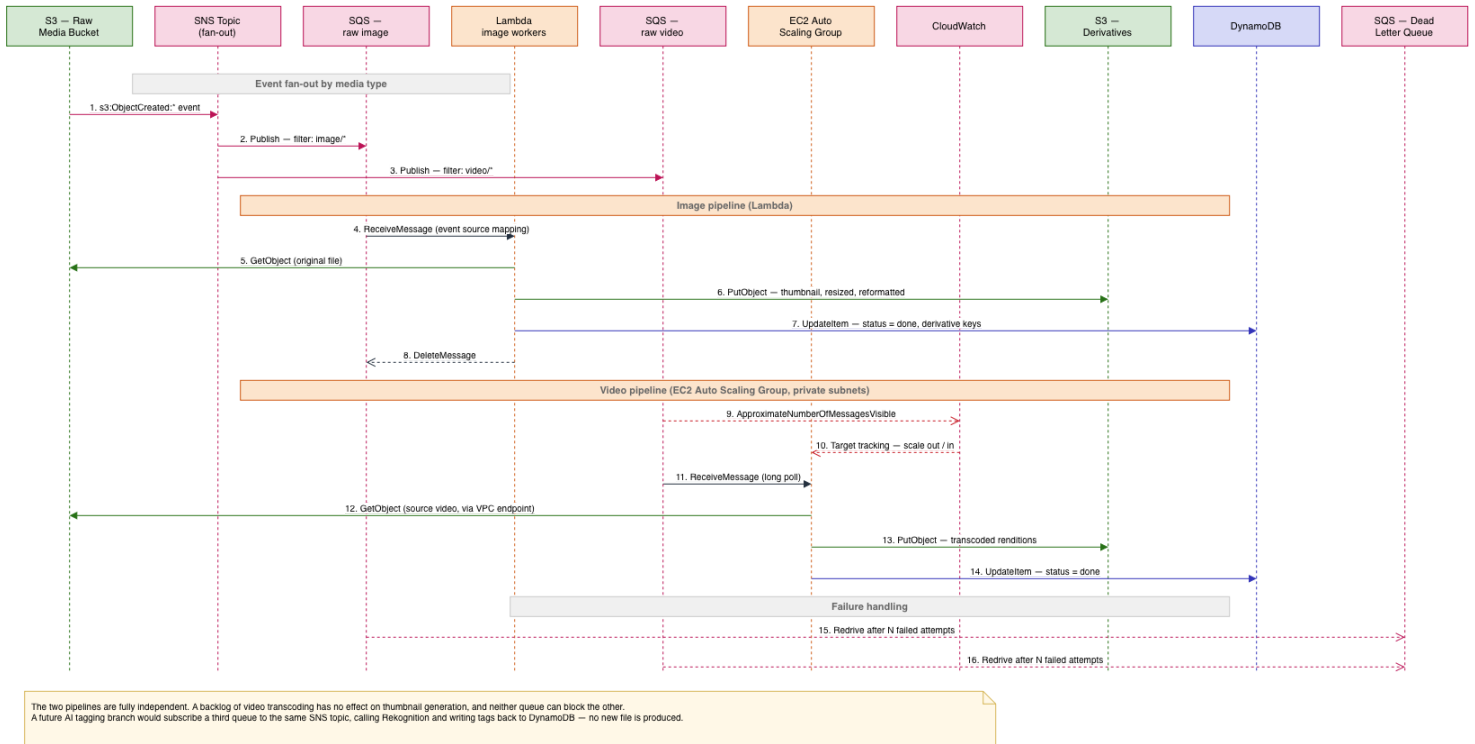


Fig. 3. Media Processing Flow.

Processing starts from the same ObjectCreated event that closes the upload flow. The SNS topic filters the event by content type and publishes it to one of two queues, so that images and video are handled by entirely independent pipelines from this point onward.

On the image side, the SQS image queue is polled by a set of Lambda functions that generate a thumbnail, produce a resized version suitable for mobile devices, and reformat the file where required. Each function writes its output to the derivatives bucket and updates the corresponding DynamoDB record with the new object keys and a completed status. On the video side, the SQS video queue is polled by the EC2 Auto Scaling Group running inside private subnets. Each worker transcodes the source file, writes the result to the derivatives bucket, and updates DynamoDB in the same way. CloudWatch continuously reads the depth of the video queue and drives the scaling policy of the fleet, adding instances as the backlog grows and removing them as it drains.

If a job fails repeatedly on either queue, SQS redirects it to the dead-letter queue after the configured number of attempts, which prevents a single malformed file from blocking the pipeline behind it. Each worker then publishes a completion event to EventBridge, which notifies the frontend to refresh the gallery. As a planned extension, the same SNS topic can publish to a third queue feeding an AI tagging Lambda function, which would call Amazon Rekognition and write the resulting labels back into DynamoDB without producing any new file.

C. Authentication Flow

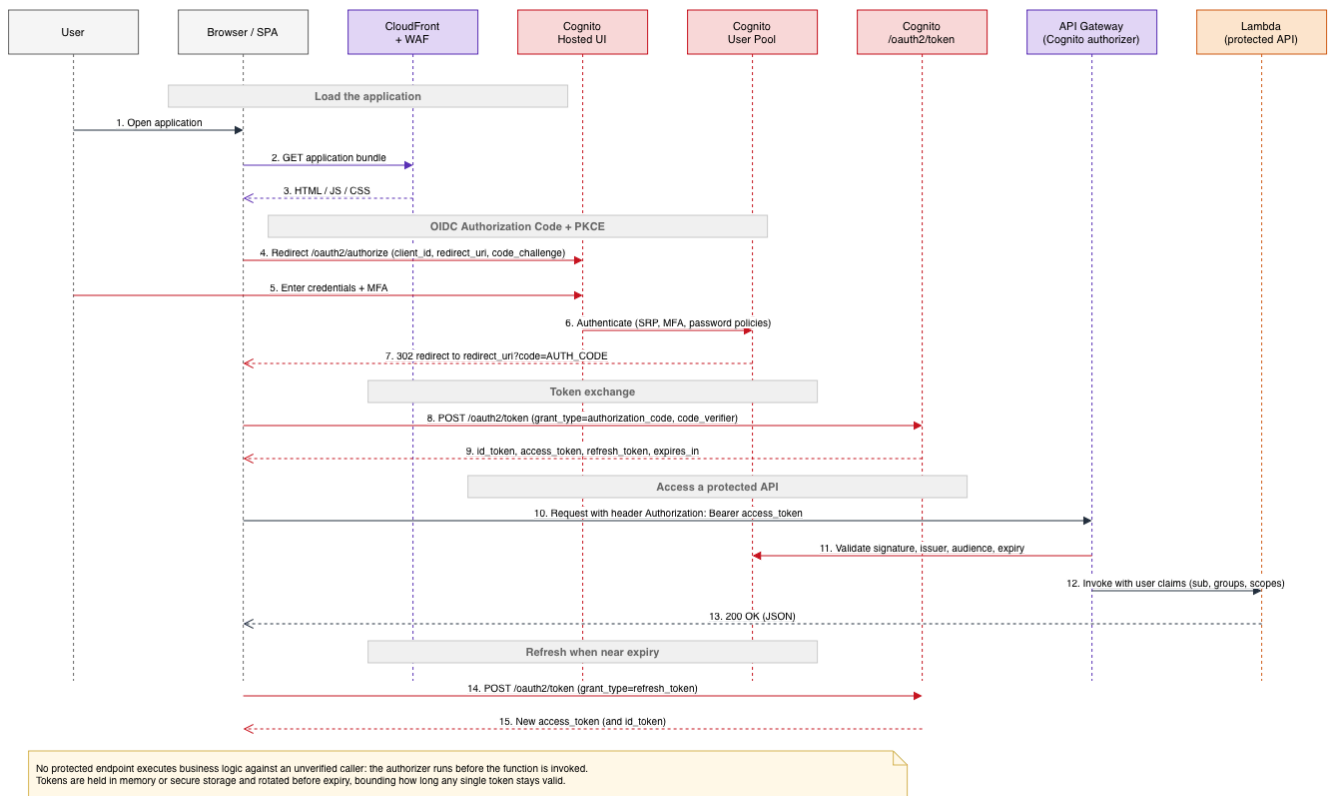


Fig. 4. Authentication Flow.

Authentication follows the OpenID Connect Authorization Code flow with PKCE, using Cognito as the identity provider. When a user opens the application, the client is directed to the Cognito Hosted UI, where the Cognito user pool performs authentication, supporting password login and multi-factor authentication. On success, Cognito returns an authorization code to the client, which the client exchanges at the token endpoint for an ID token, an access token, and a refresh token.

Every subsequent call to the API carries the access token in the Authorization header. API Gateway's Cognito authorizer validates the token's signature, issuer, audience and expiry before the request is allowed to reach any Lambda function, so that no protected endpoint executes business logic against an unverified caller. When the access token nears expiry, the client uses the refresh token to obtain a new one without asking the user to sign in again, keeping the session active while still bounding how long any single token remains valid.

D. Content Delivery Flow

A. Media Upload Flow — UML Collaboration Diagram

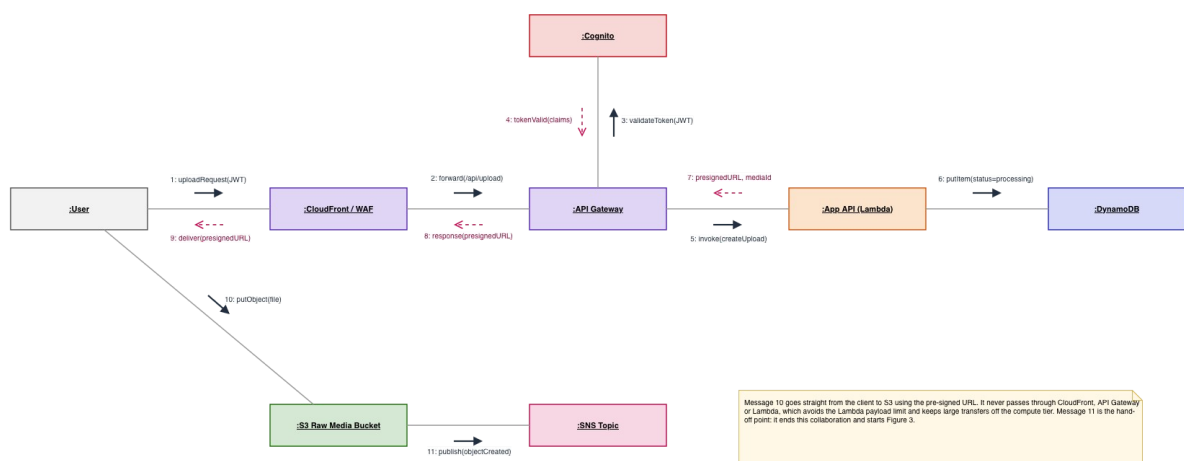


Fig. 5. Content Delivery Flow.

When a user opens the dashboard or requests a specific photo or video, the client first calls the App API through API Gateway to retrieve the relevant metadata from DynamoDB, including the S3 keys of the requested derivatives. The Lambda function returns this metadata as a set of CloudFront-accessible URLs rather than direct S3 links, since the derivatives bucket is not publicly reachable.

The client then requests the object from CloudFront using those URLs. If the object is already cached at the nearest edge location, CloudFront serves it immediately. Otherwise CloudFront retrieves it from the derivatives bucket through Origin Access Control, which permits CloudFront to read the bucket while blocking any direct public access, caches a copy at the edge, and returns it to the user. Subsequent requests for the same object from users in that region are then served entirely from the edge cache until the entry expires, which is the mechanism behind the architecture's global response time improvement.

IV. DESIGN JUSTIFICATION AND EVALUATION

A. Performance and Scalability

Performance and scalability are addressed by combining fully managed serverless components with an event-driven message pipeline, allowing the system to adapt its capacity to real-time demand rather than relying on manually provisioned resources [1].

1) Auto-scaling and event-driven design:

- **API Gateway** and **Lambda** (App API) replace the original always-on EC2 web tier entirely. Every incoming request is handled as a discrete, stateless invocation, so the compute layer scales from zero to thousands of concurrent requests without pre-provisioning, directly resolving the original CPU bottleneck.
- **S3** provides effectively unlimited storage for both the raw media and derivatives buckets, with cost driven purely by object size rather than provisioned capacity, meeting the requirement for demand that doubles every six months.
- **DynamoDB**, running in on-demand capacity mode, automatically adjusts read/write throughput as traffic grows, removing the need to forecast capacity units in advance [2].
- **SNS fan-out** with per-type **SQS** queues decouples the upload event from the processing workload, allowing the image and video pipelines to scale independently of one another.

2) Performance optimisation:

- **CloudFront**, positioned behind Route 53 and WAF, caches both the static frontend and processed media derivatives at edge locations worldwide, directly resolving the requirement that global response times outside Australia are currently poor [7].
- **Pre-signed S3 URLs** allow the client to upload media directly to the raw media bucket, bypassing API Gateway and Lambda for the payload itself, avoiding Lambda's payload-size and execution-duration constraints [3].
- **Lambda**, used for image transformation, scales concurrently with upload volume and executes only for the duration of each transformation, keeping cost proportional to actual processing work rather than idle capacity [1].

By separating the always-elastic serverless tier from the compute-intensive video workload, the architecture keeps steady-state utilisation predictable while accommodating unplanned traffic spikes without manual intervention.

B. Reliability and Availability

The design separates presentation, business logic, event distribution, and processing into independently failing units, so that the failure of one component does not cascade into the rest of the system [5].

1) Fault tolerance and redundancy:

- **S3** stores both raw and derivative media with eleven nines of durability, replicated across a minimum of three Availability Zones, so no single storage failure can result in data loss [6].
- **DynamoDB** replicates data automatically across Availability Zones within the region, providing durability and read availability without additional configuration [2].
- **SNS fan-out** means the image and video processing branches run as fully independent pipelines: a backlog or failure in video transcoding has no effect on thumbnail generation, and vice versa.
- **The EC2 Auto Scaling Group** for video workers runs across multiple Availability Zones in private subnets. If an Availability Zone becomes unavailable, the group automatically replaces lost capacity in the remaining zones [4].

2) Monitoring and recovery:

- **CloudWatch** monitors the `ApproximateNumberOfMessagesVisible` metric on the raw-video SQS queue and drives a target-tracking Auto Scaling policy, adding EC2 workers when the backlog grows and removing them as it drains [4].
- **CloudTrail** records every API call across the account, giving operators the ability to reconstruct exactly what happened, when, and by whom in the event of an incident.
- **CloudFormation** defines the entire stack as code, so a failed or misconfigured environment can be torn down and redeployed identically rather than repaired by hand, reducing recovery time and eliminating configuration drift.

3) Global availability:

- **CloudFront** serves cached content from the nearest edge location, so a regional outage at the origin does not necessarily interrupt delivery of already-cached content to users elsewhere [7].
- **Route 53** performs DNS resolution with health checks, so traffic is only routed to healthy endpoints [8].
- **AWS WAF**, positioned in front of CloudFront, filters malicious traffic before it can degrade application availability for legitimate users.

Together, these mechanisms give the architecture graceful degradation rather than full outage under partial failure: a fault in video processing, a single Availability Zone, or a burst of malicious traffic each affects only its own layer, while the core upload and viewing experience continues to function.

C. Security Design

Security is frequently the most overlooked area of a cloud design. This architecture therefore implements protection across several layers, including identity management, data encryption, and activity logging throughout the infrastructure.

1) Identity and role management:

- **Cognito** manages user sign-up and authentication using JWT tokens via User Pools. After API Gateway validates the JWT, the App API Lambda function generates temporary pre-signed S3 URLs, enabling end users to upload media directly to the raw bucket without exposing internal infrastructure or requiring full AWS credentials. [9].
- **IAM** provides JSON policies enforcing least privilege across back-end services, restricting each Lambda function, DynamoDB table, SQS queue and Rekognition worker to only the minimum permissions required for its specific task [10].

Principal	Action	Resource
App API Lambda role	dynamodb:Query, PutItem	table/photo-metadata
App API Lambda role	s3:PutObject (presign only)	raw-bucket/*
Image worker role	s3:GetObject	raw-bucket/*
Image worker role	s3:PutObject	derivatives-bucket/*
Image worker role	dynamodb:UpdateItem	table/photo-metadata
EC2 instance profile	sqs:ReceiveMessage, DeleteMessage	raw-video queue
EC2 instance profile	s3:PutObject	derivatives-bucket/*

2) Front-end and edge protection:

- **CloudTrail** enables post-incident forensics and operational audits by logging activity across the entire infrastructure, while CloudWatch provides operational observability by triggering alarms for key metrics [10].
- **WAF** and **CloudFront** provide front-end protection at the edge by preventing common web attacks, while caching media files at the nearest points of presence to offer minimum latency to users outside Australia and mitigate DDoS attacks [11].
- **Route 53** distributes load through global DNS and health checks, directing international requests to the nearest edge locations to improve performance and availability [11].

3) Data encryption:

- **At rest:** AWS Key Management Service (KMS) serves as the centralised cryptographic authority, managing customer master keys using Hardware Security Modules and AES-256 envelope encryption, ensuring data remains unreadable in the event of unauthorised physical or logical disk access [12].

- **In transit:** Transport Layer Security (TLS 1.2/1.3) is enforced along all communication pathways to prevent eavesdropping, tampering and man-in-the-middle attacks, and VPC subnets communicate exclusively over encrypted AWS internal endpoints, ensuring all internal event-driven interactions are encrypted [12].

D. Cost Optimization

To estimate infrastructure costs, several baseline assumptions regarding user growth and behaviour must be established. Starting from 5,000 users and doubling every six months, the user base is expected to reach approximately 20,000 by the end of Year 1, 80,000 by Year 2, and 320,000 by Year 3. On a monthly average, each user uploads 5 photos (2 MB each) and 0.5 videos (50 MB each), initiates 10 viewing sessions requesting 20 media items each, with 70% of traffic originating in Australia and 30% internationally.

Table I gives the estimated monthly cost at the end of each year, based on the AWS Pricing Calculator for the ap-southeast-2 (Sydney) region. Year 2 and Year 3 figures scale the Year 1 usage inputs proportionally to the projected fourfold annual growth, with tiered-pricing adjustments applied where applicable — most notably Cognito's free tier of 50,000 monthly active users, which keeps its Year 1 cost at zero despite linear growth in every other row.

TABLE I. ESTIMATED MONTHLY COST PER SERVICE AT THE END OF YEARS 1–3

Service	Configuration applied for Year 1	Y1	Y2	Y3
S3 (Raw + Derivatives + Frontend)	Storage $\approx$ 700 GB/month; 3,100,000 PUT/GET requests from event triggers.	\$21.50	\$86.00	\$344.00
CloudFront	Data transfer out $\approx$ 8,000 GB/month; 10 million HTTPS requests.	\$650.00	\$2,600.00	\$10,400.00
API Gateway	HTTP API: 3 million requests/month (upload triggers + metadata fetches).	\$3.00	\$12.00	\$48.00
Lambda (App API + image workers)	300 ms average duration; 256 MB image / 128 MB API; 10 million invocations/month.	\$15.50	\$62.00	\$248.00
DynamoDB	On-demand: 3 million WRU + 3 million RRU; $\sim$ 50 GB storage.	\$12.00	\$48.00	\$192.00
SNS & SQS (event routing)	5 million SNS publishes; 10 million SQS requests (raw image, raw video, DLQ).	\$6.50	\$26.00	\$104.00
EC2 Auto Scaling Group (video transcoding)	Baseline $2 \times$ t3.medium at 50% duty; scales on ApproximateNumberOfMessagesVisible.	\$35.00	\$140.00	\$560.00
Route 53 & WAF	1 hosted zone, 1 million DNS queries; 1 Web ACL with 3 custom rules.	\$26.00	\$28.00	\$35.00
Cognito	20,000 monthly active users (first 50,000 MAU are free).	\$0.00	\$165.00	\$1,485.00
CloudWatch & CloudTrail	10 custom metrics, 5 alarms, 10 GB log ingestion.	\$15.00	\$45.00	\$120.00
Overall monthly cost		\$784.50	\$3,212.00	\$13,536.00

CloudFormation, EventBridge and KMS cost under \$5/month combined at this scale and are omitted from Table I for clarity. VPC endpoints for S3 and SQS access from the EC2 workers add only a small fixed hourly charge. Amazon Rekognition is a planned future extension (see Section V-B) and has not yet been costed.

1) Initial resource limitations for cost control:

To prevent unpredictable billing spikes during initial deployment, strict resource limits are enforced at the application level. Video transcoding on EC2 is the most compute-intensive and costly operation in the pipeline, so the system enforces a hard limit of five minutes per video, ensuring FFmpeg jobs do not exhaust memory or cause excessive scale-outs. Users are also initially restricted to a maximum of three videos per account. This quota acts as a financial safeguard, keeping infrastructure costs predictable while the business evaluates engagement and revenue during Year 1.

This cap is a business policy rather than a technical constraint: unlike Lambda's fifteen-minute execution ceiling, EC2 has no inherent runtime limit, so the five-minute threshold can be revised once real usage data is available. Enforcement occurs before a video reaches the processing pipeline. App API validates duration at the point a client requests a pre-signed upload

URL; if the client-reported duration exceeds the cap, no URL is issued and the upload is rejected before any S3 storage or SQS message is created. As a defence against a client that misreports duration, the EC2 worker also runs a lightweight fprobe check as the first step of each job — before invoking FFmpeg's transcoding, which is the expensive part of the job — and marks the item as rejected in DynamoDB if the actual duration exceeds the cap, avoiding the cost of a full transcode on an over-limit file. As usage data accumulates, the cap can be raised in stages, or replaced by a tiered quota tied to subscription level, without any change to the upload or processing flow described in Sections II and III.

2) Cost versus benefit justification:

While projected costs scale to over \$13,000 monthly by Year 3, this expense is justified by the benefits of the serverless, event-driven model. The transition to on-demand DynamoDB and Lambda converts capital expenditure into operational expenditure, meaning the business pays for exact usage rather than funding idle servers during low-traffic periods. The EC2 Auto Scaling Group governed by SQS queue depth ensures the expensive transcoding workers are provisioned only when a backlog exists, directly answering the requirement to keep compute utilisation within the optimal 50–60% range. Finally, the majority of the cost is CloudFront data transfer, which translates directly into the low-latency global experience that drives user retention. The linear relationship between cost and user growth demonstrates that the architecture is financially sustainable and scales proportionally with business success.

E. Operational Excellence and Sustainability

Automation, observability and sustainable improvement are core tenets of the AWS Well-Architected Framework, demanding that the system self-manage, recover rapidly from failure, and minimise human error.

1) Infrastructure as code and automation:

- **CloudFormation** provides the full architecture as versioned templates, standardising environment setup, avoiding configuration drift, and making environment composition reproducible [13].
- **EventBridge and CI/CD pipelines** drive workflows independent of the main media pipeline. Integrated with GitHub, code changes deploy immediately with execution logging and client notification on state change.
- **Lambda** powers stateless business logic, image processing and AI tagging; processing statelessly enables seamless updates and automatic scaling [14].

2) Monitoring, logging and access control:

- **CloudWatch and CloudTrail** centralise logs and monitoring across distributed Lambdas, SQS queues and EC2 workers, tracking queue depth in real time to drive target-tracking Auto Scaling while capturing API activity for audit [15].
- **Access control and cost allocation** are managed through least-privilege IAM roles and specialised human IAM groups, with multi-account segregation via AWS Organizations, lifecycle rules, and tagging metadata stored in DynamoDB to simplify resource visualisation and cost attribution.

3) Sustainability:

- **Compute efficiency:** Lambda consumes energy only while code is actively running, scaling dynamically with demand and dropping to near zero at minimum usage [14]. Rather than pre-allocating an instance that runs constantly, the target-tracking EC2 Auto Scaling group scales on the number of waiting items in the SQS queue and terminates instances as the queue drains.
- **Storage and delivery efficiency:** Managed multi-tenant services such as S3 and DynamoDB maximise resource sharing, and raw media follows lifecycle policies transitioning to Infrequent Access or Glacier over time [16]. Caching frontend assets and derivatives at CloudFront edge locations drastically reduces redundant origin calls and long-haul network transfer [17].

F. Alternative Solutions and Quantitative Evaluation

Four architectural decisions were evaluated against the core business requirements — cost efficiency, operational overhead, horizontal scalability and performance. Table II summarises the quantitative comparison; the reasoning for each is set out below.

TABLE II. QUANTITATIVE COMPARISON OF ALTERNATIVE SOLUTIONS

Comparison	Key quantitative finding	Decision
EC2 vs. Lambda	300+ instances required to serve 320,000 users at 50–60% utilisation; EC2 scale-out takes minutes versus millisecond Lambda concurrency.	Lambda for the web/API tier; EC2 retained solely for video transcoding.

Comparison	Key quantitative finding	Decision
Aurora RDS vs. DynamoDB	Aurora exceeds \$3,000/month by Year 3 versus ~\$192/month for DynamoDB; ~20 ms versus single-digit-millisecond queries.	DynamoDB — roughly 14–15× cheaper with lower latency.
ALB vs. API Gateway	At ~3 million requests/month, ALB costs ~\$100/month due to fixed hourly and LCU fees versus under \$40/month for API Gateway.	API Gateway — cheaper, instantly elastic, native JWT authorisation.
Amplify vs. CloudFormation	CloudFormation incurs \$0 added service fees (<\$5/month combined with EventBridge/KMS) versus \$5,000+/month for an equivalent Amplify deployment driven by data pass-through fees.	CloudFormation — full declarative control, no abstraction lock-in.

1) Amazon EC2 versus AWS Lambda:

Continuous EC2 instances run around the clock and incur cost regardless of incoming traffic, whereas Lambda follows a pay-as-you-go model that eliminates idle compute cost under fluctuating traffic [18]. EC2 scaling relies on Auto Scaling groups requiring several minutes to boot new virtual machines during peak load, while Lambda scales concurrently and near-instantaneously across AWS-managed pools spanning multiple Availability Zones. EC2 also requires ongoing patching, security maintenance and manual multi-AZ configuration, all of which Lambda handles automatically. Under the existing architecture, six t3.micro instances run at approximately 80% utilisation to serve 5,000 users; supporting the projected 320,000 users at the target 50–60% utilisation would require over 300 instances, which is neither financially nor operationally feasible.

2) Amazon Aurora RDS versus Amazon DynamoDB:

Relational options such as Aurora scale vertically or through read replicas, which becomes prohibitively expensive as traffic grows, whereas DynamoDB scales horizontally through key-value partitioning and is far better suited to absorbing demand that doubles every six months [19]. Aurora requires provisioned capacity and therefore incurs cost during off-peak hours, while DynamoDB on-demand charges strictly per request and per byte stored, aligning database cost with application usage. The application relies on simple key-based queries by photo, user and album identifier with no multi-table joins, so DynamoDB's flexible schema accommodates media metadata and tag arrays without relational overhead.

3) Application Load Balancer versus Amazon API Gateway:

An Application Load Balancer carries a fixed hourly cost plus data-processing charges, making it cost-effective only at sustained volumes of hundreds of millions of requests per month. At the projected initial volume, API Gateway is significantly cheaper per request. ALB also requires configuring target groups, health checks and custom CloudWatch alarms, whereas API Gateway uses AWS-managed shared pools for near-instant request handling and offers native rate throttling, integrated monitoring and built-in Cognito JWT authorisation.

4) AWS Amplify versus AWS CloudFormation:

Amplify is optimised for rapid frontend prototyping through high-level abstractions, which limits control and increases complexity when composing multi-service event pipelines. CloudFormation provides declarative, low-level control over all resource properties, ensuring strict environment reproducibility across staging and production [20]. Critically, Amplify cannot natively model the decoupled backend this design requires — S3 event notifications, SNS fan-out topics, dedicated SQS redrive queues, and EC2 Auto Scaling worker fleets — so CloudFormation is used to represent the full multi-tier architecture as code without vendor abstraction lock-in.

V. ASSUMPTIONS AND LIMITATIONS

A. Achievements in This Assignment

This assignment re-architected the original EC2-based photo album application into a fully event-driven, serverless architecture capable of meeting the client's core requirements. The redesigned system replaces the always-on web server with an elastic API Gateway and Lambda compute layer, eliminating the original CPU bottleneck while ensuring the business pays only for actual usage. Media storage and delivery were re-engineered around S3 and CloudFront, directly resolving the poor response times previously experienced outside Australia, while DynamoDB replaces the relational database with a lower-cost, auto-scaling alternative better suited to the application's access patterns.

A key achievement is the automated, decoupled media processing pipeline: S3 upload events are routed through SNS fan-out into dedicated SQS queues for images and video, allowing each media type to be processed independently and enabling the pipeline to extend toward future capabilities such as AI-based photo tagging without redesigning the core flow.

For video, the architecture makes a deliberate and justified exception to the serverless-first approach, using an EC2 Auto Scaling Group governed by SQS queue depth to handle transcoding workloads that exceed Lambda's execution and memory

limits, demonstrating that the principle of matching platform to workload was applied rather than defaulting uniformly to serverless services.

Beyond the architecture itself, this assignment delivered a complete, quantified cost model. Using the AWS Pricing Calculator, monthly infrastructure costs were estimated at each stage of a three-year growth projection from 20,000 to 320,000 users, showing that costs scale proportionally and predictably rather than requiring large upfront capital investment. Combined with the enforced upload limits as an initial financial safeguard, the resulting model demonstrates that the proposed architecture is not only technically sound but also financially sustainable and appropriately scoped to the business's risk tolerance during early growth.

B. Suggestions for Future Development

While the proposed architecture fulfils all immediate business and technical requirements, several enhancements can further evolve the platform as media volume and user adoption expand.

- **Advanced search and media retrieval:** integrate Amazon OpenSearch Service alongside DynamoDB to enable full-text indexing, metadata querying and similarity-based media retrieval as the photo library grows.
- **Workflow orchestration and monitoring:** introduce AWS Step Functions into the media processing pipeline for stateful workflow orchestration, automated error handling, and visual monitoring of long-running video jobs.
- **Monetisation and storage sustainability:** implement fine-grained access tiers and usage-based quotas via API Gateway and Cognito to enable premium subscription models, and refine storage lifecycle policies by integrating S3 Intelligent-Tiering to dynamically optimize storage costs based on changing access patterns.
- **Personalisation and global resilience:** incorporate Amazon Personalize for tailored media recommendations, and evolve the deployment into a multi-region active-active architecture to further minimise global latency and increase cross-region fault tolerance.

VI. GROUP CONTRIBUTIONS

Student name	Contribution	Tasks
Le Bao Ngoc	25%	Architecture design; UML sequence and collaboration diagrams; service descriptions
Tran Quoc Toan	25%	Performance and scalability; reliability and availability; cost optimisation; achievements in this assignment
Dao Le Hong Duc	25%	Diagram overview; references; abstract and introduction
Hoang Le Quang Anh	25%	Security design; operational excellence; sustainability; alternative solutions; quantitative evaluation; future development

REFERENCES

- [1] Amazon Web Services, "Performance Efficiency Pillar," AWS Well-Architected Framework, 2024. [Online]. Available: <https://docs.aws.amazon.com/wellarchitected/latest/performance-efficiency-pillar/welcome.html>
- [2] Amazon Web Services, "On-demand capacity mode," Amazon DynamoDB Developer Guide, 2024. [Online]. Available: <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/on-demand-capacity-mode.html>
- [3] Amazon Web Services, "Download and upload objects with presigned URLs," Amazon S3 User Guide, 2024. [Online]. Available: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/using-presigned-url.html>
- [4] Amazon Web Services, "Scaling policy based on Amazon SQS," Amazon EC2 Auto Scaling User Guide, 2024. [Online]. Available: <https://docs.aws.amazon.com/autoscaling/ec2/userguide/as-using-sqs-queue.html>
- [5] Amazon Web Services, "Reliability Pillar," AWS Well-Architected Framework, 2024. [Online]. Available: <https://docs.aws.amazon.com/wellarchitected/latest/reliability-pillar/welcome.html>
- [6] Amazon Web Services, "Data protection in Amazon S3," Amazon S3 User Guide, 2024. [Online]. Available: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/DataDurability.html>
- [7] Amazon Web Services, "How CloudFront delivers content," Amazon CloudFront Developer Guide, 2024. [Online]. Available: <https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/HowCloudFrontWorks.html>
- [8] Amazon Web Services, "Creating Amazon Route 53 health checks," Amazon Route 53 Developer Guide, 2024. [Online]. Available: <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/dns-failover.html>
- [9] Amazon Web Services, "What is Amazon Cognito?," Amazon Cognito Developer Guide, 2024. [Online]. Available: <https://docs.aws.amazon.com/cognito/latest/developerguide/what-is-amazon-cognito.html>
- [10] Amazon Web Services, "AWS Well-Architected Framework: Security Pillar," AWS Whitepaper, 2021. [Online]. Available: <https://d1.awsstatic.com/whitepapers/architecture/AWS-Security-Pillar.pdf>
- [11] Amazon Web Services, "AWS Best Practices for DDoS Resiliency," AWS Whitepaper, 2022. [Online]. Available: <https://d1.awsstatic.com/SMB/aws-best-practices-ddos-resiliency-smb-security-resource.pdf>
- [12] Amazon Web Services, "AWS Key Management Service Cryptographic Details," AWS Security Whitepaper, 2021. [Online]. Available: <https://d1.awsstatic.com/whitepapers/KMS-Cryptographic-Details.pdf>
- [13] M. Wurster, U. Breitenbücher, M. Falkenthal, F. Leymann, and A. Saar, "Essential characteristics of infrastructure as code," *IEEE Software*, vol. 37, no. 1, pp. 11–16, 2020.
- [14] E. Jonas et al., "Cloud programming simplified: A Berkeley view on serverless computing," *arXiv preprint arXiv:1902.03383*, 2019.
- [15] M. Usman, S. Ferlin, A. Brunstrom, and J. Taheri, "A survey on observability of distributed edge and container-based microservices," *IEEE Access*, vol. 10, pp. 86904–86919, 2022.
- [16] A. Beloglazov, J. Abawajy, and R. Buyya, "Energy-aware resource allocation heuristics for efficient management of data centers for cloud computing," *Future Generation Computer Systems*, vol. 28, no. 5, pp. 755–768, 2012.
- [17] R. Buyya, A. Beloglazov, and J. Abawajy, "Energy-efficient management of data center resources for cloud computing: A vision, architectural elements, and open challenges," *arXiv preprint arXiv:1006.0308*, 2010.
- [18] G. Adzic and R. Chatley, "Serverless computing: Economic and architectural impact," in *Proc. 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, 2017, pp. 884–889.
- [19] G. DeCandia et al., "Dynamo: Amazon's highly available key-value store," in *Proc. 21st ACM SIGOPS Symp. Operating Systems Principles (SOSP)*, 2007, pp. 205–220.
- [20] A. Sartor, L. M. Stocchero, and A. C. da Silva, "Infrastructure as code: A systematic mapping study," *Automated Software Engineering*, vol. 28, no. 2, pp. 1–34, 2021.